

PROJECT REPORT

Machine-Learning-Based Adaptive Transmission Rate Selection in IEEE 802.11 Networks

Implementation, dataset, model methodology, evaluation, limitations, and future work

Prepared: 5 October 2026

Evidence: project source code, saved model metadata, and evaluation CSV files

This report describes the implementation and recorded results available in the project artifacts. Performance figures are historical saved metrics, not guarantees for unseen networks.

Report guide

- 1. Abstract and keywords
- 2. Introduction and problem statement
- 3. Objectives and scope
- 4. Architecture and workflow
- 5. Dataset and preprocessing
- 6. Features and target labels
- 7. Training methodology
- 8. Evaluation results
- 9. Application and operation
- 10. Privacy, limitations, and future work
- 11. Reproducibility recommendations
- 12. Conclusion and project evidence

1. Abstract

This project implements a supervised machine-learning workflow to recommend a wireless transmission-rate class from measured IEEE 802.11 link conditions. The model uses seven features: RSSI, SNR, packet loss, latency, throughput, channel utilization, and current transmission rate. Four classifiers are compared, and the saved evaluation identifies Gradient Boosting as the leading candidate. The stored reports give 98.27% test accuracy and a 98.11% macro F1-score across 1,500 test examples. A Streamlit dashboard supports local Wi-Fi metadata scanning, manual measurement entry, predictions, evaluation views, dataset upload, retraining, and CSV export. The recommendation is advisory and does not configure the wireless adapter. The saved artifacts do not record a complete training-run manifest, so the exact source-dataset provenance for the historical metrics should be confirmed before claiming reproducibility.

Keywords

IEEE 802.11; Wi-Fi; supervised learning; classification; Gradient Boosting; rate selection; RSSI; SNR; throughput; network diagnostics.

2. Introduction

Wireless link conditions can vary with signal strength, interference, packet loss, latency, channel activity, device capabilities, and distance from an access point. The transmission rate affects the trade-off between sending data quickly and maintaining reliable communication. A fixed rate may not be appropriate for every link condition.

This project studies a data-driven approach: learn a mapping from wireless measurements to a labeled recommended-rate class, then present a prediction for supplied measurements. It is designed as a local diagnostic and research tool rather than a driver-level rate controller.

2.1 Problem statement

Given a vector of observed wireless link measurements, predict one rate label from the classes supported by the trained model. Display the recommendation alongside the input values, predicted class probabilities, and available evaluation information.

2.2 Motivation

Machine-learning classifiers can represent non-linear relationships between input measurements and rate labels. Comparing several algorithms provides a way to evaluate alternatives on the available labeled dataset.

3. Objectives and scope

3.1 Objectives

- Prepare compatible wireless datasets for supervised classification.
- Train and compare multiple classifiers using a consistent test split.
- Select a model using macro F1-score and store its evaluation outputs.
- Predict a supported rate class from supplied measurements.
- Present scanning, summaries, results, retraining, and export controls in a web dashboard.

3.2 Included scope

- Local collection of some nearby-network metadata through operating-system tools.
- Manual entry of link measurements not exposed by the scanner.
- Prediction among the four stored target classes: 6, 12, 24, and 36.
- Local training and evaluation using compatible CSV or Excel data.

3.3 Excluded scope

- Changing adapter settings or forcing the predicted rate.
- Connecting to protected networks or bypassing network security.
- Guaranteeing real-world improvements to throughput, latency, or reliability.
- Treating scan results as a complete set of model measurements.

4. System architecture

Layer	Responsibility / implementation
Data sources	Adaptive Excel data, legacy CSV files, uploaded data, and local OS scan output.
Preprocessing	Schema handling, numeric conversion, row filtering, de-duplication, feature selection, and prediction-row construction.
Training	Scikit-learn pipelines compare Random Forest, Decision Tree, Logistic Regression, and Gradient Boosting.
Model artifacts	Fitted Joblib pipeline in models/best_model.pkl; metadata in models/model_metadata.json.
Web interface	Streamlit dashboard in app/web_app.py.
Wi-Fi scanner	Windows netsh or Linux nmcli integration in src/wifi_scanner.py.
Evaluation	Comparison, classification report, and confusion matrix under results/metrics/.

4.1 Prediction workflow

1. The user selects a detected network or chooses manual input.
2. Real wireless measurements are entered for required fields.
3. The dashboard constructs a one-row DataFrame in the model feature layout.
4. The loaded pipeline predicts one class and returns class probabilities.
5. The dashboard displays the class, maximum probability as confidence, health summary, and history.
6. The recommendation remains informational; the app does not change the adapter.

5. Dataset and preprocessing

The adaptive Excel dataset contains 5,000 rows and 11 columns, including seven measurement fields and a recommended_transmission_rate target. Its target distribution is 1,760 rows for 6 Mbps, 1,267 for 12 Mbps, 1,158 for 24 Mbps, and 815 for 36 Mbps.

Dataset	Observed dimensions / description
WiFi_Transmission_Rate_Recommendation_Dataset_5000.xlsx	5,000 rows x 11 columns; includes the adaptive target field.
dataset_complete.csv	124,980 raw rows x 20 columns; legacy experiment data.
dataset_P_A_auto.csv	3,979 raw rows x 17 columns; legacy experiment data.

For the adaptive schema, preprocessing converts feature and target fields to numeric values, removes rows missing required values, drops duplicates, and selects the seven features. Separate paths handle legacy schemas. Legacy conversion can use derived fields or zero for unavailable channel utilization; those assumptions should be identified if that path is used for a reported run.

The training script lists all three data files as defaults and loads those whose paths exist. The stored test support is 1,500, consistent with a 30% split of 5,000 rows, but the saved metric files do not identify the precise source file list used when those metrics were produced. Do not assert exact dataset provenance without a training log or manifest.

6. Features and target

Feature	Description	Unit
rsi	Received signal strength indicator	dBm
snr	Signal-to-noise ratio	dB
packet_loss	Percentage of packets not received	percent
latency	Round-trip or link delay measurement	milliseconds
throughput	Observed received data rate	Mbps
channel_utilization	Share of channel activity in use	percent
current_transmission_rate	Current rate supplied for the sample	Mbps
recommended_transmission_rate	Classification target	6, 12, 24, or 36 Mbps

The classifier learns the target labels present in its training data. A predicted label should not be described as a direct physical measurement or as a rate automatically applied to a Wi-Fi adapter.

7. Training methodology

7.1 Split and selection

The source code uses train_test_split with a 30% test fraction, stratification by target, and random_state=42. Candidate models are evaluated on the split created during that run. The saved classification report has 1,500 examples. Candidate selection sorts macro F1-score in descending order.

7.2 Candidate classifiers and configuration

Classifier	Configuration in source
Random Forest	200 trees; random_state=42; n_jobs=-1.
Decision Tree	max_depth=16; random_state=42.
Logistic Regression	max_iter=1000; random_state=42.
Gradient Boosting	200 estimators; random_state=42.

Each classifier is wrapped in a Scikit-learn Pipeline with StandardScaler followed by the classifier. The selected pipeline is serialized with Joblib. The current requirements pin scikit-learn to 1.8.0 to match the saved model artifact and support compatible deserialization.

8. Evaluation results

8.1 Candidate-model comparison

Model	Accuracy	Macro F1
Gradient Boosting	98.27%	98.11%
Random Forest	97.73%	97.64%
Decision Tree	96.33%	96.26%
Logistic Regression	95.60%	95.53%

8.2 Selected model summary

Selected classifier: Gradient Boosting. Recorded accuracy: 0.9826667 (98.27%). Recorded macro F1: 0.9810816 (98.11%). Test support: 1,500 examples. The rounded accuracy corresponds to 1,474 correct and 26 incorrect predictions.

8.3 Per-class report

Class	Precision	Recall	F1	Support
6 Mbps	98.86%	98.86%	98.86%	528
12 Mbps	98.42%	98.42%	98.42%	380
24 Mbps	99.12%	96.83%	97.96%	347
36 Mbps	95.65%	98.78%	97.19%	245
Macro avg	98.01%	98.22%	98.11%	1,500
Weighted avg	98.29%	98.27%	98.27%	1,500

Precision is the share of predictions for a class that are correct. Recall is the share of actual examples of that class identified correctly. F1 is the harmonic mean of precision and recall.

8.4 Confusion matrix

Actual / predicted	6 Mbps	12 Mbps	24 Mbps	36 Mbps
6 Mbps	522	6	0	0
12 Mbps	6	374	0	0
24 Mbps	0	0	336	11
36 Mbps	0	0	3	242

The matrix contains 1,474 correct predictions on its diagonal and 26 errors off the diagonal. Recorded errors are between 6 and 12 Mbps, and between 24 and 36 Mbps.

8.5 Feature importance

Feature	Stored importance	Approx. share
SNR	0.546580	54.66%
Throughput	0.271726	27.17%
Packet loss	0.077813	7.78%
Channel utilization	0.047386	4.74%
RSSI	0.028507	2.85%
Latency	0.027930	2.79%
Current transmission rate	0.000057	0.0057%

Importance values describe how this fitted model used features. They are model-specific and do not establish causation or universal importance across environments.

8.6 Interpretation

The stored reports indicate strong classification on the test examples associated with those artifacts. They do not establish equivalent performance on different devices, routers, locations, future traffic, or a production network. No independent field trial is included in the saved evaluation evidence.

9. Application and operation

9.1 Dashboard capabilities

- Scan button and optional automatic network scanning every 30 seconds.
- Detected SSID, BSSID, signal, strength, channel, and security display where the operating system reports them.
- Input controls for all seven model features.
- A separate dashboard link-health score.
- Predicted class, per-class probabilities, and maximum class probability shown as confidence.
- Prediction history visualization and CSV export.
- Model comparison, classification report, confusion matrix, and uploaded-data retraining.

9.2 Launch steps

1. Install project dependencies from requirements.txt.
2. From the project root, run: `streamlit run app/web_app.py`
3. Open `http://localhost:8501` in a browser.
4. Scan for nearby Wi-Fi or select manual input.
5. Enter actual values for measurements not exposed by the scanner.
6. Review the recommendation and export a result if required.

9.3 Scanner behavior

Windows scanning calls `netsh wlan show networks mode=bssid`. Linux scanning calls `nmcli` and requires `NetworkManager`. Scanner output depends on the local operating system and driver. The scanner does not automatically provide all seven model fields. If hosted remotely, scanning describes the server host's wireless environment, not necessarily the browser user's device.

10. Important interpretation distinctions

Test accuracy is a dataset-level score. The dashboard confidence is the largest predicted probability for one input sample. Probability calibration has not been separately evaluated, so an individual displayed confidence is not a guarantee of correctness.

The dashboard's health score is a separate weighted calculation and is not model accuracy. Default values displayed by the app should not be described as live measurements unless independently verified. Recommendations depend on the quality and provenance of input values.

11. Privacy, safety, and limitations

11.1 Privacy and safe use

- Normal scanning and prediction are designed for local use and do not require a remote prediction service.
- Network identifiers and exported history may be sensitive; inspect files before sharing.
- The system does not change adapter settings, connect to protected networks, or force hardware changes.

11.2 Limitations

- The saved accuracy describes a recorded holdout evaluation, not a real-world guarantee.
- Metric files do not preserve exact dataset file names, hashes, training timestamp, or full package-version manifest for the historical training event.
- The training script can load all configured default data files that exist; a fresh run can therefore use a different sample set than the one associated with the stored 1,500-example report.
- Legacy schema conversions may derive fields or substitute unavailable channel utilization.
- The scanner does not provide all model features; manual measurements are required.
- The saved model supports four labels only: 6, 12, 24, and 36.
- The provided artifacts do not record cross-validation, independent test data, probability calibration, or a field trial.
- Feature importance is not a causal explanation.
- The app recommends a class but does not apply it to hardware.

12. Reproducibility and recommended validation

For a defensible report, record the exact source dataset list, row counts, file hashes, code revision, package versions, training command, split parameters, and generated metric files. The source code sets a fixed random seed, but exact reproducibility also requires matching data and software versions.

Recommended validation steps

1. Choose and document the dataset files; do not rely on implicit file discovery.
2. Keep a test set separate from model tuning where possible.
3. Record model parameters, random seed, software versions, and dataset hashes.
4. Save per-class metrics, class counts, and the confusion matrix for every run.
5. Evaluate on independently collected data.
6. Test multiple devices, access points, distances, interference conditions, and loads.
7. Measure actual network outcomes before claiming rate or reliability improvement.
8. Compare recommended behavior with a baseline under equivalent conditions.

13. Future work

- Collect diverse real-world data and document provenance.

- Add a complete model-training manifest and model version.
- Evaluate on environment-independent and temporally independent data.
- Assess probability calibration before interpreting confidence.
- Acquire currently manual metrics automatically where platform support permits.
- Measure operational outcomes, not only label-prediction accuracy.
- Clarify which inputs are scanned, manually entered, defaulted, or unavailable.
- Review all legacy transformations for alignment with target semantics.

14. Conclusion

The project combines data preprocessing, classifier comparison, saved model artifacts, local Wi-Fi scanning, and a Streamlit dashboard. The saved comparison selects Gradient Boosting, reporting 98.27% accuracy and 98.11% macro F1 on 1,500 test examples. These results support a statement about the recorded evaluation only. Dataset provenance and independent real-world validation are required before generalizing the metrics. The current application should be described as an advisory prototype that predicts one of four labels and does not apply changes to network hardware.

15. Project artifacts used

Artifact	Purpose
models/model_metadata.json	Selected model, feature list, classes, scores, and feature importance.
results/metrics/model_comparison.csv	Saved candidate accuracy and macro F1.
results/metrics/classification_report.csv	Per-class and aggregate precision, recall, F1, and support.
results/metrics/confusion_matrix.csv	Actual-versus-predicted class counts.
src/train.py	Candidate definitions, train/test split, scoring, selection, and saving.
src/preprocessing.py	Dataset loading, schema handling, feature preparation, and prediction inputs.
src/wifi_scanner.py	Local Windows/Linux wireless metadata scanning.
app/web_app.py	Streamlit UI, prediction flow, history, evaluation, and retraining.
data/WiFi_Transmission_Rate_Recommendation_Dataset_5000.xlsx	Adaptive labeled dataset.
requirements.txt	Python dependencies and scikit-learn compatibility pin.

Appendix A. Metrics summary

Measure	Recorded value
Selected model	Gradient Boosting
Accuracy	0.9826667 (98.27%)
Macro F1	0.9810816 (98.11%)
Weighted F1	0.9826872 (98.27%)
Test support	1,500
Correct / incorrect	1,474 / 26
Output classes	6, 12, 24, 36

End of report.